

— PROJECT OVERVIEW —

PDLC_DEMO — Project Overview

Program	PainEase PCA Advanced (PP3500) – patient-controlled analgesia infusion pump, with a surrounding connectivity adapter and Cloud Suite
Regulatory path	510(k) with a Predetermined Change Control Plan (PCCP)
Last updated	2026-04-21

AGENDA

How we'll walk through this.

1**Project Overview & Strategy****2****Agentic Approach — Project
Shape, Structure & Capabilities****3****Quality & Process — How the
Guardrails Work****4****Agentic Approach — Solving
Complex Problems with Humans in
Charge****5****The Project Console****6****Appendix — Quick links**

1

Project Overview & Strategy

§1.1

What PP3500 is

PP3500 — **PainEase PCA Advanced** — is a patient-controlled analgesia infusion pump. The program as modeled in this repo is a **single composite system** of SaMD + SiMD + custom medical-electrical hardware, with adjacent infrastructure around it:

DHF (ARCHITECTURE NAME)	CLASSIFICATION	COMPOSITION	WHAT IT DOES
PCA Device (pca-device)	Class II medical device	SiMD pump firmware + SaMD modules + hardware	On-pump delivery, dose-limit enforcement, alarms, UI, self-test
Connectivity Adapter (connectivity-adapter)	MDDS (non-device post-2015 FDA)	SaMD	On-prem bridge that carries PCA telemetry + drug-library pushes to/from the Cloud Suite
Cloud Suite (cloud-suite)	Mixed — see children	SaMD + non-device software	Platform DHF. Seven child DHFs carry the regulatory weight individually

Cloud Suite children (seven nested DHFs):

§1.1

What PP3500 is (cont.)

CHILD DHF	CLASSIFICATION	ROLE
drug-library-manager	Class II SaMD (PCA accessory — directly mutates the dose-limit table the pump enforces)	Drug library authoring, signing, and push
fleet-management	Non-device	Pump inventory, assignment, firmware-update distribution
compliance-reports	Non-device	Administrative reporting
analytics-dashboard	Non-device	Operational analytics
inventory-tracker	Non-device	Consumables and asset tracking
alerts-engine	TBD — see regulatory strategy	Threshold-based alarm escalation
clinical-interface	Non-device	Clinician-facing administrative UI

§1.2

Regulatory strategy at a glance

We are filing **one 510(k)** for the PCA device itself, **with an integrated PCCP** that pre-authorizes two classes of post-market change: drug-library updates and bounded firmware updates.

Filing scope = PCA device alone

The Connectivity Adapter (MDDS) and all Cloud Suite components except the Drug Library Manager are **out of the PP3500 filing**. Each is analyzed and filed (or not filed) on its own regulatory merits. See `docs/project/strategies/regulatory-strategy.md` §1 *Filing Scope: PCA Device Alone*.

Critical-requirement carve-out (CtS / CtF / CtC / CtP)

Every user need and design input is tagged with one or more of **Critical to Safety / Function / Compliance / Performance**. Only Ct*-tagged requirements are in scope for the 510(k) + PCCP; everything else ships post-clearance on the commercial track. This keeps the submission minimum and the V&V burden tractable. See `regulatory-strategy.md` §1 *Filing Strategy — Critical-Requirement Carve-out*.

PCCP envelope

The PCCP pre-authorizes changes to Ct*-tagged requirements that stay within pre-specified bounds — drug-library updates, firmware patches against a fixed risk profile, and predictive-alarm SaMD updates that meet the PCCP change-protocol acceptance criteria.

Predicate

PP3000 (K190567) — a prior PainEase PCA pump in the same family. The PCCP envelope is the substantive delta vs. the predicate.

Drug Library Manager

, classified as Class II SaMD accessory, is either bundled into the PP3500 filing or filed separately (posture TBD in submission planning — see `docs/project/submissions/510k/composition-manifest.md`).

VARIATION A · SCOPE ICEBERG

Regulatory strategy at a glance

Filing scope = PCA device alone

Requirement carve-out (CtS / CtF

PCCP envelope

Predicate

Drug Library Manager

↑ FILING SCOPE

↓ COMMERCIAL / OUT-OF-SCOPE

IN FILING

visible to FDA

OUT OF FILING

commercial · separate filings · post-clearance

The Connectivity Adapter

We are filing **one 510(k)** for the PCA device itself, **with an integrated PCCP** that pre-authorizes two classes of post-market change: drug-library updates and bounded firmware updates.

§1.3

Key deliverables

Q-Sub package

[docs/project/submissions/qsub/] — cover letter, device description, classification validation, PCCP scope questions.

510(k) submission

[docs/project/submissions/510k/] — substantial-equivalence argument to PP3000, software docs, performance and validation data, risk analysis, labeling. Composition driven by `composition-manifest.md`.

PCCP document

[docs/project/submissions/pccp/] — change categories, modification protocols, performance criteria, reporting plan.

Trace matrix

per DHF [docs/project/dhfs/<dhf>/design-controls/trace-matrix/] — User Needs ↔ Design Inputs ↔ Architecture ↔ V&V ↔ Risk, with a **Filing Scope** column derived from Ct* tags.

2

Agentic Approach — Project Shape, Structure & Capabilities

§2.1

Project shape & structure

Flat multi-DHF layout mapped to IEC 62304 § 5 (one software system composed of software items), with Cloud Suite as a parent DHF whose children are independently classified:

Ten DHFs total (three top-level, seven nested under Cloud Suite). A composition manifest per submission lists which DHF pieces roll into that filing — filings are not 1:1 with DHFs. The PP3500 510(k) uses only `pca-device` (lead) + `connectivity-adapter` (cybersecurity-only pull) + optionally `drug-library-manager` (bundled-vs-standalone TBD).

§2.2

Operating rules baked into the project

Task-first workflow (hard gate)

A PreToolUse hook (`.claude/hooks/check-active-task.sh`) denies any Edit/Write/NotebookEdit unless the session has an active task. Every substantive change in the repo traces back to a task document under `tasks/<person>/NNN-*.md`.

One task, one file

All design work, analysis, drafts, and phase deliverables produced under a task belong inside that task's numbered markdown file — never as sibling `NNN-task-p1.md` files. If a deliverable genuinely lives elsewhere (a new doc in `docs/`), the task still records what was produced and links to it.

Strategy / Lessons capture backstops

UserPromptSubmit + Stop hooks detect strategic-intent signals and block ending a session that generated strategy/lessons content without capturing it into the active task doc using `←!— STRATEGY CONTENT: domain, topics →` or `<!-- LESSONS LEARNED: category → blocks. /strategy` and `/lessons` harvest from those blocks.

Session-start security check

The `project-secops` agent runs `security-assert.sh` on every session against the `security.*` allowlists in `project.yml`. Results cached to `tasks/<person>/SECOPS.md` with a 7-day TTL. High-severity drift triggers guided remediation.

Docflow entry enforcement

A PreToolUse Bash hook (`block-direct-conversion.sh`) refuses direct `pandoc / unzip / soffice / pdftotext` calls so all PDF/DOCX/XLSX conversions round-trip through the audited `/docflow` pipeline.

`project.yml` as single source of truth

Identity, device family, portfolio context, DHF topology (with `regulatory + filing + parent` per DHF), team roster, skill registries, secops allowlists, and `advisors.enabled` curation all live in one manifest.

Skills (project-level)

15 skills are installed under `.claude/skills/` and allow-listed in `project.yml` under `security.approved_skills`, plus four document-format skills (`docx`, `pptx`, `xlsx`, `pdf`) from the `anthropic` builtin registry. The project-specific and highest-leverage ones:

SKILL	WHAT IT DOES
task	Task lifecycle: create / find / list / update / show. Owns the task-first gate and Strategy/Lessons capture hooks.
medtech-docs	Scaffolds <code>docs/</code> tree, manages DHFs, imports FDA guidance / ISO-IEC standards / industry frameworks, generates compliance dashboard.
docflow	Round-trip conversion between markdown and DOCX / DOC / PDF / XLSX with image fidelity, cross-reference resolution, and metadata tracking. Entry enforced by hook.
strategy	Harvests <code>← — STRATEGY CONTENT: domain →</code> blocks from tasks into unified shared strategy docs across eight domains (regulatory, commercial, architecture, development, testing, risk, postmarket, operations).
tracker	Builds the submission-package tracker from architecture + regulatory strategy + composition manifests; renders HTML dashboard.
trace-matrix	Bidirectional design-controls trace (UN ↔ DI ↔ Architecture ↔ V&V ↔ Risk) per DHF with project-adaptive parsers. Emits <code>.md</code> deliverable + <code>.json</code> sidecar consumed by the console.
advisors	Installs and curates the 11 Core Team Assistant personas + 2 advisory panels + KOL personas. Same definitions feed Claude Code and the console chat.
project-console	Scaffolds and maintains the local FastAPI console (see §5).
lessons	Harvests <code>← — LESSONS LEARNED →</code> blocks into the ledger; promotes mature lessons to their permanent home (skill / CLAUDE.md / agent / rule / glossary).
best-practices	Audits project setup against checks from the shared hitachi registry and each skill's <code>## Best Practices</code> table.
secops	Session-start security posture check, attestations, allowlists.
change-control	(Scaffold only.) Bridge to Confluence + Comala/SoftComply for Part 11 review, and Windchill for release vault. Hybrid freeze-point lifecycle (draft → frozen → released).
sync-skills	Bidirectional sync with the shared <code>hitachi</code> registry (<code>../hitachi</code> local checkout).
skill-creator	Authors new skills, runs evals.
digest	Morning briefing + on-demand append to <code>CHANGELOG.md</code> .

Agents (persona advisors)

13

PROJECT OVERVIEW

§2.4 · AGENTS (PERSONA ADVISORS)

23 domain agents total — a mix of **Core Team Assistants** and **Key Opinion Leaders** — plus project-secops on the security side. All live under `.claude/agents/` (Core Team) and `tools/project-console/agents/` (KOLs, project-specific). Each is exposed identically to (a) Claude Code via `Agent(subagent_type=...)` and (b) the project console as chat advisors. `project.yml` under `advisors.enabled` curates which Core Team assistants are on for a given session (today: `regulatory-affairs`, `clinical-affairs`, `risk-management`).

AGENT	ROLE FOCUS
<code>regulatory-affairs</code>	510(k) pathway, substantial-equivalence argument, predicate selection, PCCP scoping, standards mapping
<code>clinical-affairs</code>	Clinical evidence strategy, KOL engagement, user-needs validation
<code>systems-engineering</code>	Architecture, requirements decomposition, trace integrity across DHFs
<code>rd-lead</code>	Engineering execution, design-output quality, feasibility
<code>vnv-lead</code>	V&V strategy, test protocol authoring, evidence sufficiency
<code>risk-management</code>	ISO 14971 risk analysis, benefit-risk, risk controls
<code>human-factors</code>	IEC 62366 use-related risk, summative usability
<code>quality-engineering</code>	ISO 13485, design controls compliance, audit readiness (FDA + Notified Body)
<code>cybersecurity</code>	Threat model, SBOM, IEC 81001-5-1, FDA §524B pre/post-market controls
<code>post-market</code>	Surveillance, complaint handling, PSUR/PMSR, CAPA feedback loop
<code>program-manager</code>	Schedule, scope, stakeholder alignment
<code>core-team-panel</code>	PM + RA + Clinical + QE + R&D — program-level decisions
<code>design-review-panel</code>	Systems + R&D + V&V + HFE + Risk + QE — technical gate reviews
<code>project-secops</code>	Session-start security posture audit against <code>project.yml</code>

§ 2 . 4

Agents (persona advisors) (cont.)

KOL	FOCUS
James E. Paul	PCA pump safety, acute pain management — primary PP3500 anchor KOL
Kathleen K. Giuliano	IV smart pump usability, medication administration safety, human factors
Lisa Gorski	Infusion nursing standards, vascular access
Priyadarshini Pennathur	Human factors engineering, medical device usability
Evan Kirkendall	Smart pump safety, pediatric medication errors, clinical informatics
Parth Shah	Smart pump alert fatigue, medication safety
Sini Kuitunen	Smart pump dosing limits, pediatric / neonatal medication safety
Susan Braithwaite	Insulin infusion protocols, ICU glucose management (portfolio context, not PCA-specific)
kol-panel-pp3500	Round-robin panel of the four most PCA-relevant KOLs
kol-panel-pp3500-llm	Same four KOLs with a silent LLM moderator picking the next speaker based on conversation

3

Quality & Process — How the Guardrails Work

§3.1

Skills — reusable, expert-grade playbooks

A **skill** is a packaged, named way to do a recurring, complex job. Think of it as a senior-engineer checklist turned into a one-command action, so every team member does the job the same way every time. Instead of each person inventing their own approach to “create a new design history file” or “convert a QMS SOP from DOCX to markdown,” they run `/medtech-docs add-dhf` or `/docflow import` and the skill handles the details — folder layout, naming conventions, image extraction, metadata capture.

SKILL	WHAT IT AUTOMATES
<code>/task</code>	Every change the team makes must be tied to a named task document. This skill creates it, tracks it, closes it out.
<code>/medtech-docs</code>	Scaffolds the documentation tree per FDA and IEC 62304 expectations — so no one has to guess where a file goes.
<code>/docflow</code>	Converts formal documents (DOCX/PDF/XLSX) to markdown and back, preserving images, cross-references, and metadata. Round-trips are reproducible.
<code>/tracker</code>	Builds the 510(k) + PCCP submission checklist automatically from design files and strategy documents — eliminates the “lost spreadsheet” problem.
<code>/trace-matrix</code>	Rebuilds the User-Needs → Design-Inputs → Architecture → V&V → Risk trace on demand — the backbone of any design-controls audit.
<code>/best-practices</code>	Runs a standing audit of the project and raises flags when something drifts.

Hooks — automatic tripwires you can't forget to run

A **hook** is a small automation that fires at a specific moment — when a session starts, when a file is about to be edited, when work is about to end — and either allows, blocks, or adds context to the action. Hooks are invisible until one trips. When one trips, it means the system caught an issue before it became a problem.

HOOK	WHEN IT FIRES	WHAT IT PREVENTS
Task-first gate (check-active-task.sh)	Before any file edit	Refuses to edit a file unless there's an active task document that owns the change. No orphan edits, ever.
Session security check (security-assert.sh)	When a session opens	Runs the project-secops audit against project.yml allowlists. High-severity drift triggers guided remediation.
Session env (session-env.sh)	When a session opens	Exports CLAUDE_SESSION_ID so the task gate can key per-session state.
Strategy/lessons capture (capture-signals.sh + capture-check.sh)	UserPromptSubmit + Stop	Arms when a prompt shows strategic-intent signals; blocks session end if the armed task didn't capture the decision into a <!-- STRATEGY CONTENT --> / <!-- LESSONS LEARNED --> block.
Docflow entry (block-direct-conversion.sh)	Before raw pandoc / unzip / soffice / pdftotext Bash calls	Refuses direct conversion commands that would bypass the audited /docflow pipeline.
Session cleanup (session-cleanup.sh)	When a session ends	Removes per-session state files so completed sessions don't leave orphans.

Think of them as the central interlock of the project. They're the guardrails you about seatbelts.

Rules — project conventions the tool reads and obeys

A **rule** is a short written convention stored in `CLAUDE.md` or `.claude/rules/` that the AI assistant reads before acting. Rules are how we encode agreements like “every folder under `docs/` must have a README” or “every README has a `## Conventions` and a `## Changelog` section” — and then the assistant actually follows them, every time, without being reminded.

One task, one file

All design work, analysis, and drafts produced under a task live inside the task’s numbered markdown file — never as ad-hoc sibling files. Keeps the trail auditable.

READMEs have `## Conventions` and `## Changelog` sections

When Claude edits a README, it appends a changelog row explaining the rationale — AI sessions collapse many edits into one commit, and the changelog captures context git alone doesn’t.

Capture strategy and lessons in real time

Strategic decisions go into the active task as `←!— STRATEGY CONTENT: domain →` blocks in the same turn they were made — not as a deferred cleanup pass. Same rule for lessons.

Never fabricate standard, clinical, or regulatory content

Anything not derivable from a distilled source under `docs/external/` or `docs/internal/source-md/` is flagged `[VERIFY]` inline.

Demo banner on every fabricated datum

Every illustrative clinical figure, placeholder predicate, or sample analysis carries `_Demo sample data — not for clinical use._` near the top.

§3.4

What this adds up to

Every substantive change in this repository is, by construction:

Owned

by a named, dated task document (task gate).

Located

in a folder whose rules were read before the write (README-before-write).

Traced

to a design input, architecture node, V&V item, and risk control (trace matrix).

Audited

against a standing best-practices registry (/best-practices).

Security-checked

at session start (secops hook).

Captured

for strategic/lessons content before any session ends (capture hooks).

VARIATION A · CONCEPT CANVAS

What this adds up to

**Owned**

by a named, dated task document (task gate).

Located

in a folder whose rules were read before the write (README-before-write).

Traced

to a design input, architecture node, V&V item, and risk control (trace matrix).

Audited

against a standing best-practices registry (`/best-practices`).

4

Agentic Approach — Solving Complex Problems with Humans in Charge

§4.1

What an "agent" actually is here

An agent (or "advisor") is a persona — Regulatory Affairs, Clinical Affairs, Systems Engineering, V&V Lead, Risk Management, Human Factors, Quality Engineering, Cybersecurity, Post-Market Surveillance, R&D Lead, Program Manager — configured to:

Read only the documents a real person in that role would read

— FDA guidance, ISO/IEC standards, the project's own design controls, the predicate analysis, the relevant DHF.

Answer within the scope of that role

and say "that's outside my lane — ask the XX advisor" when a question crosses a boundary.

Cite the sources

for every claim, so the human reviewer can verify.

Flag disagreement

with a draft rather than rubber-stamp it.

VARIATION A · CONCEPT CANVAS

What an "agent" actually is here

**Read only the documents a real person in that role would read**

— FDA guidance, ISO/IEC standards, the project's own design controls, the predicate analysis, the relevant DHF.

Answer within the scope of that role

and say "that's outside my lane — ask the XX advisor" when a question crosses a boundary.

Cite the sources

for every claim, so the human reviewer can verify.

Flag disagreement

with a draft rather than rubber-stamp it.

§4.2

How it helps with complex problems

Regulated medical device work is not a single-discipline problem. A single design decision — say, “should the Connectivity Adapter push drug-library deltas over mTLS, or should the pump pull signed bundles on demand?” — simultaneously touches:

Cybersecurity

(IEC 81001-5-1, signing-trust model, FDA §524B)

Risk management

(what happens if the push fails mid-update? ISO 14971 hazard analysis)

Systems engineering

(where the SaMD / MDDS boundary sits; impact on composition manifest)

Regulatory

(does this change PCCP envelope scope? Does it alter the MDDS classification of the adapter?)

Clinical

(does it introduce dose-limit latency a clinician must account for?)

V&V

(what new test cases does it require? Do they belong in filing-scope or commercial V&V?)

§4.3

Humans decide. Agents advise.

Every regulatory decision on this program — pathway choice, predicate selection, classification, risk acceptability, design-input approval, V&V sufficiency, release readiness — is made by a named human and recorded in a named task document. The agent's role is to:

Brief the human

with a structured analysis grounded in specific sources.

Surface counterpoints

(every advisor runs a "counterpoint pass" — what would the opposing argument be?).

Refuse to fabricate

when evidence isn't there — advisors are instructed to say "the source doesn't support this" rather than invent.

Leave a paper trail

of what it read, what it said, and when.

§ 4 . 4

Where the handoff happens

The AI never occupies step 3 or step 6. That is not a limitation — that is the design.

STEP	WHO	WHAT
1. Draft analysis	Agent	Reads grounded sources, drafts structured answer with citations and counterpoints.
2. Review & redirect	Human	Reviews the analysis, asks follow-ups, rejects or redirects.
3. Decision	Human	Chooses the path forward and writes it into the task document.
4. Implementation	Human + agent	Human directs; agent drafts changes; task-first gate ensures every edit is owned and traced.
5. Verification	Human (or panel)	Independent panel agent reviews before the change is finalized.
6. Sign-off in QMS of record	Human only	Controlled artifact is promoted; signatures applied by named humans in the system of record.

The AI never occupies step 3 or step 6. That is not a limitation — that is the design.

VARIATION A · HANDOFF RELAY

Where the handoff happens

01

AGENT

Reads grounded sources, drafts structured answer with citations and counterpoints.

02

HUMAN

Reviews the analysis, asks follow-ups, rejects or redirects.

03

HUMAN

Chooses the path forward and writes it into the task document.

04

HUMAN + AGENT

Human directs; agent drafts changes; task-first gate ensures every edit is owned and traced.

05

HUMAN (OR PANEL)

Independent panel agent reviews before the change is finalized.

06

HUMAN ONLY

Controlled artifact is promoted; signatures applied by named humans in the system of record.

The AI never occupies step 3 or step 6. That is not a limitation — that is the design.

§4.5

Why this works for regulated work

Traditional AI skepticism in regulated environments boils down to: "I can't put an AI decision in front of a Notified Body or the FDA." Correct. We don't. What we put in front of the regulator is:

- A human-signed decision.
- In a task document owned by a named person.
- Backed by a trace matrix rebuilt from source.
- Supported by a structured analysis with cited sources.
- Vetted by multiple specialist perspectives.
- Produced inside a toolchain whose compliance properties are enforced by hooks, not hope.

5

The Project Console

§5.1

Landing page

![Landing](http://127.0.0.1:8765/)

Branded with the **GlobalLogic** theme pack (scraped and materialized by `/project-console` theme). Top-level tiles for **Agents** (23 domain agents), **Documents**, **Dashboards**, and **Trace Matrix**. Workflows tile marked "Coming soon." <http://127.0.0.1:8765/>

§5.2

Agents (persona advisors)

![Agents](http://127.0.0.1:8765/agents)

23 agents total

— 11 Core Team Assistants + 8 KOLs + 4 panels (2 Core Team + 2 KOL). A filter toggle switches between **All**, **Key Opinion Leaders**, and **Core Team Assistants**.

Each card links to a full threaded chat UI with grounding sources visible

Core Team curation is driven by `project`

yml → advisors.enabled`.

<http://127.0.0.1:8765/agents>

§5.3

Documents explorer

![Documents](http://127.0.0.1:8765/documents)

File tree over the entire project with markdown rendering in the right pane

Surfaces DHFs, submissions, standards, FDA guidance, tasks, and strategies side by side

A unified Assistant drawer (task ben/024) mounts on this page: the user picks an advisor, the drawer grounds on the document currently open, and thread history persists per-scope in browser localStorage

Junk-filtered (macOS Icon\r, `\_\* , Windows Thumbs.db` hidden).

<http://127.0.0.1:8765/documents>

§5.4

Dashboards

! [Dashboards] (http://127.0.0.1:8765/dashboards)

Submission Package Tracker

— docs/project/submissions/submission-tracker.html (510[k] + PCCP deliverable tracker)

§5.5

Trace Matrix

!Trace matrix](http://127.0.0.1:8765/trace-matrix)

Each DHF gets a bidirectional trace sidecar driven by `trace-matrix.yml` at repo root. The index shows which DHFs have a built matrix (today: **pca-device**) and which are empty-state (today: the other nine). Empty-state rows offer a **Build this DHF** button; once adapters exist the deterministic `build.py` runs; otherwise `/trace-matrix init` authors them via the Claude Agent SDK.

§5.6

How the console relates to Claude Code

Same agents. Same grounding. Two runtimes. Updating an agent (`.claude/agents/regulatory-affairs.md`) updates both surfaces simultaneously — no drift between what Claude Code sees and what the console exposes.

SURFACE	RUNTIME	GROUNDING	AUTH
Claude Code main thread	CLI, developer workstation	Direct filesystem access	Local
project-console chat	Browser, FastAPI + Claude Agent SDK	Same agent files under <code>.claude/agents/</code> via shared <code>advisors_lib</code> symlink	OAuth to Claude CLI; domain-restricted per <code>project.yml</code> → <code>security.approved_email_domains</code>

Same agents. Same grounding. Two runtimes. Updating an agent (`.claude/agents/regulatory-affairs.md`) updates both surfaces simultaneously — no drift between what Claude Code sees and what the console exposes.

6

Appendix — Quick links